Tuning and timestreams

Overview

After VNA screening (see the VNA screening document), the tuning procedure finds the optimal tone power and frequency for each resonator. Timestreams are then taken with the tuned tones and calibrated, and the photon pulses in them are found.

Script	What it does	Saves
`03_tune.py`	Plans the tones, sweeps the tone power, and finds each tone's optimal power and frequency (with two interactive windows)	`tune{k}`
`04_stream.py`	Opens a window for previewing timestreams and taking named timestream runs	`ts{n}`
`05_calibrate_streams.py`	Calibrates selected timestreams, first in an interactive window, then in batch	`ts_cal{c}`
`06_find_photons.py`	Finds the photon pulses and cosmic rays in calibrated timestreams, with a window for choosing the thresholds	`ts_photons{p}`

As with the VNA screening scripts, each script has a settings block at the top, and command-line options override the settings for one run. Text in the `NOTES` setting (or `--notes`) is saved with the data. The scripts work on the device store `<DATA_DIR>/<COOLDOWN>/<DEVICE>.zarr`, or the store given with `--store`. With `--simulate`, the measurements are taken on the simulated device of a store made by `simulate_store.py`.

A tuning group holds the tuning measurements and the tuned tones. Each timestream run is a separate group at the store root. It saves a copy of the tones it used, so it can be analyzed without the tuning group.

```
<device>.zarr/
  tune{k}/              tuning (03)
  ts{n}/                timestream run (04); attributes: name, notes,
                        decimation stage, duration, tone sets, tuning
    set{s}/             calibration target sweep of tone set s
      write{w}/         tones written for one stream (the whole tone
                        set, or part of it)
        stream/         timestream data
  ts_cal{c}/            calibrated timestreams (05)
  ts_photons{p}/        photons and cosmic rays found in calibrated
                        timestreams (06)
```

Tone sets

Each module reads out a 500 MHz band centered on its NCO frequency (600 MHz with extended bandwidth), with at most 1024 tones. We call this band an NCO window. If the tones don't all fit in the available modules at once, they are divided into **tone sets**. A tone set is the group of tones written to the modules together, with one NCO frequency per module. A sweep of all the tones sweeps one tone set after another and combines (stitches) them into a single sweep. Each stream contains the tones of one tone set.

The tone sets are assigned once, when the tuning is planned:

- The tones are covered with as few NCO windows as possible. Window edges are placed in the widest gaps between resonances, and every tone is at least `EDGE_MARGIN` from an edge.
- Windows are assigned to the modules in order, and each pass through the modules makes one tone set.
- If extended bandwidth is enabled on the board, a window can be 600 MHz wide, with its tones in the central 575 MHz.
- `STREAM_DEC` limits each window to the number of tones that can be streamed at once at that decimation stage (currently 128 at stages 0-1 and 1024 at stages 2-6). This way the NCO frequencies never need to change during streaming.
- `WINDOWS` (`--windows`) selects a subset of the windows, to measure only part of the array. `--list-windows` prints the windows with their NCO frequencies, tone ranges, and numbers of tones.

03_tune.py: make a tuning

The script creates a new tuning, `tune{k}`, and runs these steps:

1. **plan**: Start from a resonator list (`resonators{j}`, the latest by default). Remove tones that are too close to another resonance, closer than `MIN_SEP_LW` times the larger of the two linewidths. `PAIRS` sets whether both tones of a close pair are removed, or only the tone of the shallower resonance (by the dip depths that `02_find_resonances.py` measures on the VNA sweep). Divide the remaining tones into tone sets. Optionally, add `N_CAL_TONES` off-resonance calibration tones per window (`res_idxs` -1, -2, ...). The initial tones are saved in `tones_init`, and the CRS settings, including each tone set's NCO frequencies, in `crs_config`.
2. **power_sweep**: Take a target sweep at each power in `POWERS`, stitched over the tone sets (`power_sweep/set{i}`). If the sweep is interrupted, the next run continues from the next power.
3. **power_fits**: The IQ series window fits the IQ loops at every power, then fits each resonator's nonlinearity parameter a_{nl} as a function of power (using a_{nl} values in `A_FIT_RANGE`). The power where a_{nl} reaches `ANL` is marked; Shift+click or drag the line to choose a different power. Resonators whose fit never reaches `ANL` get the median of the other chosen powers. The powers are then clipped to `ARES_LIMITS` and saved in `power_choice`.
4. **fq_sweep**: Take a stitched target sweep at the chosen powers.
5. **fq_finder**: In the fq_finder window, set each tone's frequency and sweep span, or reject the tone. If the window is closed before every tone is done, the next run continues where it left off.
6. **tones**: Save the tuned tones in `tones` (`fres`, `qres`, `ares`, `res_idxs`, `tone_set`, `tone_module`), without the rejected tones.

Steps that are already done are skipped, so the script picks up where it stopped. `--redo power_fits`, `--redo fq_finder`, or `--redo tones` repeats that step and the steps after it (`power_fits` only until the fq sweep has been taken). Sweeps are never retaken in an existing tuning; use `--new` to start a new tuning instead.

Contents of a tuning

```
tune{k}/              attributes: resonators (the resonator list that
                      res_idxs refer to)
  crs_config/         modules, analog bank, bandwidth, module settings,
                      and the NCO frequencies of each tone set
  tones_init/         initial tones and the settings used to plan them
  power_sweep/set{i}  stitched target sweep at each power (raw)
  power_fits/         IQ loop fits (analysis)
  power_choice/       chosen powers
```

```
  fq_sweep/            stitched target sweep at the chosen powers, with
                       the fq_finder results fres_opt, qres_opt,
                       reject_reason
  tones/               tuned tones
```

`tones/`, `crs_config/`, and the resonator list contain everything needed to measure with the tuning. `citkid.daq_layout.port_tuning` copies only these to another store (for example, for a new cooldown), without the sweeps.

Settings

```
SERIAL_NUMBER = 46          # CRS serial number
INTERFACE = 'enp5s0'        # Ethernet interface connected to the CRS
DATA_DIR = '~/data'         # stores are in DATA_DIR/COOLDOWN/DEVICE.zarr
COOLDOWN = 'cooldown'       # cooldown name
DEVICE = 'device'           # device name
MODULES = [1]               # modules connected to this device
# Tones
RESONATORS = None           # resonator list index (None: the latest)
MIN_SEP_LW = 5.             # remove tones within this many linewidths of
                            # another resonance
# For close pairs: 'drop_both' removes both tones; 'keep_stronger' keeps
# the tone of the deeper resonance (dip depth measured by 02)
PAIRS = 'drop_both'
EDGE_MARGIN = 10e6          # minimum distance from a tone to the edge of
                            # its NCO window, Hz
# Fastest decimation stage that will be used for streaming. Each module
# gets at most as many tones per tone set as can be streamed at once at
# this stage (currently 128 at stages 0-1 and 1024 at stages 2-6), so the
# NCO frequencies never change during streaming
STREAM_DEC = 6
# Number of off-resonance calibration tones (res_idxs < 0) per NCO window.
# They are placed in the middles of the widest gaps between resonances,
# and their timestreams are calibrated with the 'ts_offres' path
N_CAL_TONES = 0
CAL_MIN_SEP_LW = 10.        # minimum distance from a calibration tone to
                            # any resonance, in linewidths
# NCO windows to use (indices in frequency order, as printed by
# --list-windows), to measure only part of the array. None: all windows.
# For example, STREAM_DEC = 0 and WINDOWS = [2] gives the (up to 128)
# tones of one window
WINDOWS = None
# Sweeps
POWERS = np.arange(-60., -35., 3)   # power sweep, dBm
SHUFFLE = False             # randomize the order of each tone's powers
SEED = 0                    # random seed for SHUFFLE
NPOINTS_FINE = 1000         # fine sweep points per tone
NPOINTS_GAIN = 50           # gain sweep points per tone
NPOINTS_ROUGH = None        # rough sweep points per tone (None: no rough
                            # sweep)
GAIN_SPAN_FACTOR = 10       # gain sweep span = 10 x fine sweep span
NSAMPS = 200                # samples averaged per point
# Power choice
F_AVG = 4                   # number of fine sweep points averaged together
                            # in the fits
A_FIT_RANGE = (0.05, 0.8)   # a_nl range used in the a_nl vs power fits
ANL = 0.5                   # target nonlinearity parameter a_nl
```

```
# Resonators whose fit never reaches ANL get the median of the other chosen
# powers. All chosen powers are then clipped to ARES_LIMITS (dBm)
ARES_LIMITS = (-55., -35.)
# fq_finder
FRES_UPDATE_METHOD = 'spacing'  # 'none', 'spacing', 'minS21', 'distance'
RMV_GAIN_SIMPLE = True      # apply a simple gain correction to the
                            # displayed data

STEPS = ('plan', 'power_sweep', 'power_fits', 'fq_sweep', 'fq_finder',
         'tones')
REDOABLE = ('power_fits', 'fq_finder', 'tones')

# Notes saved with the data this script makes (attribute 'notes'), e.g.
# the filters and attenuators on the DAC and ADC lines. Multi-line is fine.
NOTES = """
"""
```

Command line

```
usage: 03_tune.py [-h] [--serial SERIAL] [--iface IFACE] [--data-dir DATA_DIR]
                  [--cooldown COOLDOWN] [--device DEVICE] [--store STORE]
                  [--modules MODULES [MODULES ...]] [--resonators RESONATORS]
                  [--tune TUNE] [--new] [--redo {power_fits,fq_finder,tones}]
                  [--simulate] [--windows WINDOWS [WINDOWS ...]]
                  [--list-windows] [--notes NOTES]

Tune a device.

options:
  -h, --help            show this help message and exit
  --serial SERIAL
  --iface IFACE
  --data-dir DATA_DIR
  --cooldown COOLDOWN
  --device DEVICE
  --store STORE         store path (instead of data dir/cooldown/device)
  --modules MODULES [MODULES ...]
  --resonators RESONATORS
                        resonator list index (default: the latest)
  --tune TUNE           tuning to continue (default: the latest)
  --new                 start a new tuning
  --redo {power_fits,fq_finder,tones}
                        repeat this step and the ones after it
  --simulate            sweep the store's simulated device
  --windows WINDOWS [WINDOWS ...]
                        NCO windows to use (default: all)
  --list-windows        print the NCO windows of a new tuning and stop
  --notes NOTES         notes saved with the data (quote multi-line text)
```

04_stream.py: take timestreams

The script opens a window for one tuning (by default, the latest tuning that has tuned tones). The window has two parts:

- **Preview** (not saved): Choose a tone set and press Calibrate to take a target sweep of it (kept in memory). Then press Capture to write the tones and stream for a few seconds at the chosen decimation stage. Select a tone to plot

its timestream, either as the fractional frequency shift x (the projection onto the tangent of the IQ loop at the tone frequency), as the components along and across the loop, or as I and Q. The calibration loop and the timestream are also plotted in the IQ plane.

- **Run** (saved): Enter a name, notes, a decimation stage, a duration, and the tone sets to stream, or pick a preset from `PRESETS` (presets can also choose tone sets). All tone sets are streamed by default; choosing fewer saves time, for example for a long run on one tone set. The run is saved as the next `ts{n}`. For each tone set, a new calibration target sweep is taken (`set{s}`), followed by the stream (`set{s}/write{w}/stream`). At fast decimation stages, a tone set with too many tones to stream at once is streamed in several parts, with the same NCO frequencies. The window lists the runs taken with this tuning.

Streaming uses the board's full network bandwidth, so this script needs exclusive use of the board for as long as its window is open. It takes global control of the board (to change the decimation stage) and creates a lock file on the DAQ computer (`~/.citkid/locks/crs<serial>.lock`). It won't start while another script is sweeping or streaming on the board. While its window is open, any other script that tries to use the board (`vna_quick_check.py`, `01_vna_sweep.py`, `03_tune.py`, another `04_stream.py`, ...) stops immediately with an error that names the running `04_stream.py` session. Close the window to release the board. If a session crashes, its lock is ignored.

With `--simulate`, the script streams the simulated device of a store made by `simulate_store.py`. With `SIM_PULSES = True` (or `--sim-pulses`), the simulated streams also contain photon pulses on each tone and cosmic rays on all tones at once, set by `SIM_PULSE_SETTINGS`, so that `06_find_photons.py` can be tried on them. While pulses are on, the simulated board uses the signal-to-noise ratio `SIM_SNR_DB` (80 dB by default), because the stores' default noise level hides photon-sized pulses at decimation stages 0 and 1.

Settings

```
SERIAL_NUMBER = 46          # CRS serial number
INTERFACE = 'enp5s0'        # Ethernet interface connected to the CRS
DATA_DIR = '~/data'         # stores are in DATA_DIR/COOLDOWN/DEVICE.zarr
COOLDOWN = 'cooldown'       # cooldown name
DEVICE = 'device'           # device name
MODULES = [1]               # modules connected to this device
# Preview
PREVIEW_DEC = 6             # preview decimation stage
PREVIEW_S = 5.              # preview duration, s
# Runs offered as presets (name, decimation stage, duration in s, notes)
PRESETS = [
    dict(name = 'dark_dec6', dec_stage = 6, duration_s = 600.,
         notes = 'dark, long'),
    dict(name = 'dark_dec0', dec_stage = 0, duration_s = 60.,
         notes = 'dark, fast'),
]
# Calibration sweeps (as in 03_tune.py)
NPOINTS_FINE = 1000         # fine sweep points per tone
NPOINTS_GAIN = 50           # gain sweep points per tone
GAIN_SPAN_FACTOR = 10       # gain sweep span = 10 x fine sweep span
NSAMPS = 200                # samples averaged per point
# Simulation (--simulate only): add photon pulses and cosmic rays to the
# streams. Photons arrive on each tone independently (rate in Hz, size x
# as a fractional frequency shift, decay time tau in s); cosmic rays hit
# every streamed tone at once (cosmic_rate in Hz, size cosmic_x, varying
# between tones by a log-normal factor of width cosmic_spread)
SIM_PULSES = False
SIM_PULSE_SETTINGS = dict(rate = 20., x = 2e-6, tau = 150e-6,
                          cosmic_rate = 1., cosmic_x = 1e-5,
                          cosmic_spread = 0.5)
# Signal-to-noise ratio of the simulated board while SIM_PULSES is on, dB
```

```
# (as simulate_store.py's SNR_DB). The stores' default, 40, buries
# photon-sized pulses at decimation stages 0-1. None: the store's value
SIM_SNR_DB = 80.
```

Command line

```
usage: 04_stream.py [-h] [--serial SERIAL] [--iface IFACE]
                    [--data-dir DATA_DIR] [--cooldown COOLDOWN]
                    [--device DEVICE] [--store STORE]
                    [--modules MODULES [MODULES ...]] [--tune TUNE]
                    [--simulate] [--sim-pulses]

Take timestreams.

options:
  -h, --help            show this help message and exit
  --serial SERIAL
  --iface IFACE
  --data-dir DATA_DIR
  --cooldown COOLDOWN
  --device DEVICE
  --store STORE         store path (instead of data dir/cooldown/device)
  --modules MODULES [MODULES ...]
  --tune TUNE           tuning (default: the latest with tuned tones)
  --simulate            stream the store's simulated device
  --sim-pulses          with --simulate: add photon pulses and cosmic rays to
                        the streams
```

05_calibrate_streams.py: calibrate timestreams

List the timestreams to calibrate in `CALIBRATE`. Each entry selects:

- a run of `04_stream.py`, by name (the latest run with that name) or by index (`run = n` for `ts{n}`);
- the part of the run to load, in zarr chunks along time (`chunks`: all chunks, the first n, or `(first, stop)`). Loading only a few chunks keeps the calibration fast;
- optionally, a subset of the tone sets.

If `CALIBRATE` is empty, the script lists the available runs with their names, decimation stages, durations, numbers and lengths of chunks, and notes.

The selected timestreams are loaded into one DataSet for each combination of decimation stage, number of samples, and resonator list. Each row is one tone from one run. `source_idx` records which entry and tone set the row came from, and `write_idx` records which tones were streamed together (for common-mode removal). The rows are calibrated with the 'ts' path, first in the interactive window and then in batch for the remaining rows. Calibration tones get their own DataSet, calibrated with the 'ts_offres' path.

The results are saved in a `ts_cal{c}` group at the store root. Running the script again with the same `CALIBRATE` continues in the same group; a different `CALIBRATE`, or `--new`, starts a new group.

Settings

```
DATA_DIR = '~/data'         # stores are in DATA_DIR/COOLDOWN/DEVICE.zarr
COOLDOWN = 'cooldown'       # cooldown name
DEVICE = 'device'           # device name
# Timestreams to calibrate. Each entry selects a run by name (the latest
# run with that name) or by run (its index n in ts{n}); chunks, the zarr
# chunks to load (None: all; n: the first n; (first, stop): a range); and
```

```
# tone_sets (None: all). Leave the list empty to list the runs.
CALIBRATE = [
    # dict(name = 'dark_dec6', chunks = 1),
    # dict(name = 'dark_dec0', chunks = 2),
]
F_AVG = 1                   # number of fine sweep points averaged together
INTERACTIVE = True          # review in the interactive window before the
                            # batch calibration

# Notes saved with the data this script makes (attribute 'notes'), e.g.
# the filters and attenuators on the DAC and ADC lines. Multi-line is fine.
NOTES = """
"""
```

Command line

```
usage: 05_calibrate_streams.py [-h] [--data-dir DATA_DIR]
                               [--cooldown COOLDOWN] [--device DEVICE]
                               [--store STORE] [--new] [--no-gui]
                               [--notes NOTES]

Calibrate timestreams.

options:
  -h, --help           show this help message and exit
  --data-dir DATA_DIR
  --cooldown COOLDOWN
  --device DEVICE
  --store STORE        store path (instead of data dir/cooldown/device)
  --new                start a new ts_cal group
  --no-gui             calibrate in batch only
  --notes NOTES        notes saved with the data (quote multi-line text)
```

06_find_photons.py: find photons

The script uses a stream calibration made by `05_calibrate_streams.py` (by default the latest `ts_cal{c}`; choose another with `--cal c`). In this script, a timestream is one tone set of one run, over the whole run. Any run can be searched if the calibration covers its tone set; it doesn't have to be one of the runs the calibration was fit on. The first time a run is searched, the calibration is applied to all of its samples and the calibrated x is saved in the calibration group (`ts_cal{c}/applied`), so later searches of the same run start faster.

If `TIMESTREAMS` is empty, the script lists the timestreams it can search (run, name, tone set, duration, number of tones) and stops. Otherwise, list the timestreams to search in `TIMESTREAMS`, by index (as listed) or by run name. A run name selects every tone set of the latest run with that name.

The detection timestreams are kept in memory up to `MEMORY_GB`, so the threshold window can rerun tones quickly. Beyond that, they are read from the saved calibrated x again when needed.

Photon detection

Each tone is searched with an optimal filter that is built iteratively:

1. **First pass**: the peaks of the timestream (smoothed over 3 samples) above `INIT_NSIGMA` times the noise are found. Peaks higher than `INIT_HMAX` times the median peak height are left out. This cut is loose on purpose: it only keeps large pulses, such as cosmic rays, out of the first template, so that as many photons as possible are kept.

2. **Optimal filter**: the selected pulses are averaged into a template, the noise spectrum is measured away from the pulses, and the optimal filter is built from the two. The filtered timestream gives each pulse a height in units of the mean photon pulse, so the thresholds don't depend on the tone's responsivity. The pulses with heights between `HMIN` and `HMAX` are used for the next template. This is repeated `OF_ITERATIONS` times. Optionally, `HMIN` is chosen automatically (`HMIN_FALSE_FRAC`) as the lowest height at which few of the accepted pulses are expected to be noise.

Choosing the thresholds

For the first run, a window opens after the detection of each of its tone sets (skip it with `INTERACTIVE = False` or `--no-gui`). The left side lists the tones with their number of pulses and their rate. The plots show, for the selected tone or for all tones together:

- the first-pass peak heights, in units of their median, with the `INIT_HMAX` cut;
- the heights of the filtered pulses in units of the mean photon, with the expected number of noise peaks overlaid (measured from the filter output with the sign flipped, which contains noise but no pulses), and the `HMIN` and `HMAX` lines;
- the mean pulse of the selected tone.

Drag the lines or type the values, then press Rerun (Ctrl+R). With "Apply to all tones", the values become the global thresholds, and every tone without its own thresholds is run again. With "Apply to this tone", the selected tone gets its own thresholds (marked with * in the list). "Use the global thresholds" removes them. Close the window (or press Done) to continue.

Several timestreams

When timestreams of several runs are listed, the timestreams of the first run are processed as above. `REUSE` (or `--reuse`) sets what the later runs reuse from it, tone set by tone set:

- `'settings'` (default): the first run's thresholds for the same tone set, including the ones set for single tones. The full detection is run on each later timestream, without the window.
- `'filter'`: the first run's optimal filter for each tone (matched by resonator index) and its thresholds. The filter is applied without iterating, and the pulse heights stay in units of the first run's mean photon. Tones that the first run has no filter for are detected from scratch. All timestreams must have the same decimation stage.
- `'none'`: each timestream is processed from scratch, with the window.

Coincidences and cosmic rays

After the detection, the tones of each tone write (the tones streamed together) are compared. A pulse is coincident if another tone in the same write has a pulse within `COINC_WIN` samples before or after it (2 samples by default: 52 μs at decimation stage 0). For this comparison, pulses down to `COINC_NSIGMA` times the filter noise count, so that small pulses of a cosmic ray on other tones are also found.

- **Photons**: each tone's mean photon pulse averages the pulses with heights between `HMIN` and `HMAX` that are not coincident with any other tone. Its decay time is fit.
- **Cosmic rays**: the pulses of all tones in a write are grouped into events. An event starts at the earliest pulse not yet in an event and includes every pulse up to 2 × `COINC_WIN` samples later. Events seen by at least `COSMIC_MIN_MULT` tones (3 by default) are saved as cosmic rays, with the time, the number of tones, and for each tone its pulse height (from the filter, and the raw peak of the timestream, since the filter height is biased for pulses longer than a photon pulse).

What is saved

Each run of the script makes a new `ts_photons{p}` group at the store root, with the calibration it used (attribute `source`) and one `timestream{j}` group per listed timestream:

```
ts_photons{p}/          attributes: source (ts_cal{c}), timestreams, reuse
  timestream{j}/        attributes: run, tone set, calibration sweep and
                        writes, samples, sample period, how it was
```

```
                    detected, thresholds, settings
                    per tone: res_idxs, status (empty if detected),
                    hmin, hmax, sigma_y, amp_ph (the mean photon
                    height, in x), rate, n_used, tau, template,
                    avg_pulse (mean photon pulse), psd, filter
    pulses/         every pulse found: tone, sample, height, raw peak,
                    candidate (at least hmin), in_window, mult
                    (number of tones), used (in the mean photon pulse)
    cosmic/events/  t0 (sample), time_s, mult, tone write
    cosmic/hits/    one row per tone of each event: tone, sample,
                    height (photons and x), raw peak
```

Settings

```
DATA_DIR = '~/data'         # stores are in DATA_DIR/COOLDOWN/DEVICE.zarr
COOLDOWN = 'cooldown'       # cooldown name
DEVICE = 'device'           # device name
CALIBRATION = None          # ts_cal index (None: the latest)
# Timestreams to search, by index (as listed) or run name (every tone set
# of the latest run with that name). The first run sets the thresholds;
# leave the list empty to list the timestreams.
TIMESTREAMS = [
    # 0,
    # 'dark_dec0',
]
# How the timestreams after the first reuse its work: 'settings' (its
# thresholds, full detection, no window), 'filter' (its optimal filters
# and thresholds, no iterations), or 'none' (from scratch, with the window)
REUSE = 'settings'
INTERACTIVE = True          # check the thresholds in the window
# First pass
INIT_NSIGMA = 6.            # lower threshold, noise sigma of the boxcar
INIT_HMAX = 3.              # upper cut, times the median first-pass height
# Optimal filter
OF_ITERATIONS = 5           # filter iterations
HMIN = 0.5                  # height window, in units of the mean photon
HMAX = 1.6
# Choose HMIN automatically: the lowest value (from HMIN up to
# HMIN_AUTO_MAX) with at most this fraction of expected noise pulses in the
# height window. None: use HMIN as it is
HMIN_FALSE_FRAC = None
HMIN_AUTO_MAX = 0.8
TEMPLATE_PRE_S = 0.5e-3     # template length before the pulse maximum, s
TEMPLATE_POST_S = 2.5e-3    # template length after the pulse maximum, s
HIGHPASS_HZ = 10.           # drift removal before detection, Hz
# Coincidences and cosmic rays
COINC_NSIGMA = 4.           # pulses counted for coincidences, filter sigma
COINC_WIN = 2               # coincidence window, +- samples
COSMIC_MIN_MULT = 3         # fewest tones of a cosmic ray
MEMORY_GB = 4.              # detection timestreams kept in memory, GB

# Notes saved with the data this script makes (attribute 'notes'), e.g.
# the filters and attenuators on the DAC and ADC lines. Multi-line is fine.
NOTES = """
"""
```

Command line

```
usage: 06_find_photons.py [-h] [--data-dir DATA_DIR] [--cooldown COOLDOWN]
                          [--device DEVICE] [--store STORE] [--cal CAL]
                          [--reuse {none,settings,filter}] [--no-gui]
                          [--notes NOTES]

Find photons in calibrated timestreams.

options:
  -h, --help            show this help message and exit
  --data-dir DATA_DIR
  --cooldown COOLDOWN
  --device DEVICE
  --store STORE         store path (instead of data dir/cooldown/device)
  --cal CAL             ts_cal index (default: the latest)
  --reuse {none,settings,filter}
                        how later timestreams reuse the first one
  --no-gui              skip the threshold window
  --notes NOTES         notes saved with the data (quote multi-line text)
```